A Lease-Gated Architecture for Safe and Modular Autonomous Drone Systems

Brendan Waldrop

Barrett, The Honors College and

School of Computing and Augmented Intelligence

Spring 2026

Thesis Director: Adwith Malpe

Secondary Committee Member: Dr. Ryan Meuth

Table of Contents

Abstract

This project aims to describe an open-source companion computer stack focused on deterministic safety and modular architecture for autonomous unmanned aerial vehicle (UAV) systems. The core of this architecture is based around the PX4 autopilot software, with a focus on an on-board companion computer running ROS2. Together, these tools form a capable foundation for autonomous UAV development. Existing public resources provide only enough to get started but leave a significant gap between a working example and a system with the supervision, fault handling, and architectural structure required for a production-ready system. This project seeks to bridge that gap by providing an open reference architecture and companion computer framework designed with production concerns in mind. The stack is organized into three decoupled layers – Supervisor, Control, and Decision – unified by a lease-gate mechanism that ensures autonomous operation is always explicitly authorized and never implicitly assumed. Documentation is also provided alongside the architecture, validated in simulation using Cosys-AirSim and PX4 SITL, offering a reproducible development workflow that allows researchers and developers to iterate and test without requiring physical hardware.

Introduction

Modern UAVs have emerged as a solution to a wide range of use-cases across multiple commercial sectors such as agriculture, public safety, surveying, and logistics. These applications cannot be supported by just an RC controller and drone operator alone; they require autonomous functionality and complex decision-making to effectively perform these various functions at scale. To understand where autonomy lives in these systems, it helps to first understand the basic structure of one of these UAVs.

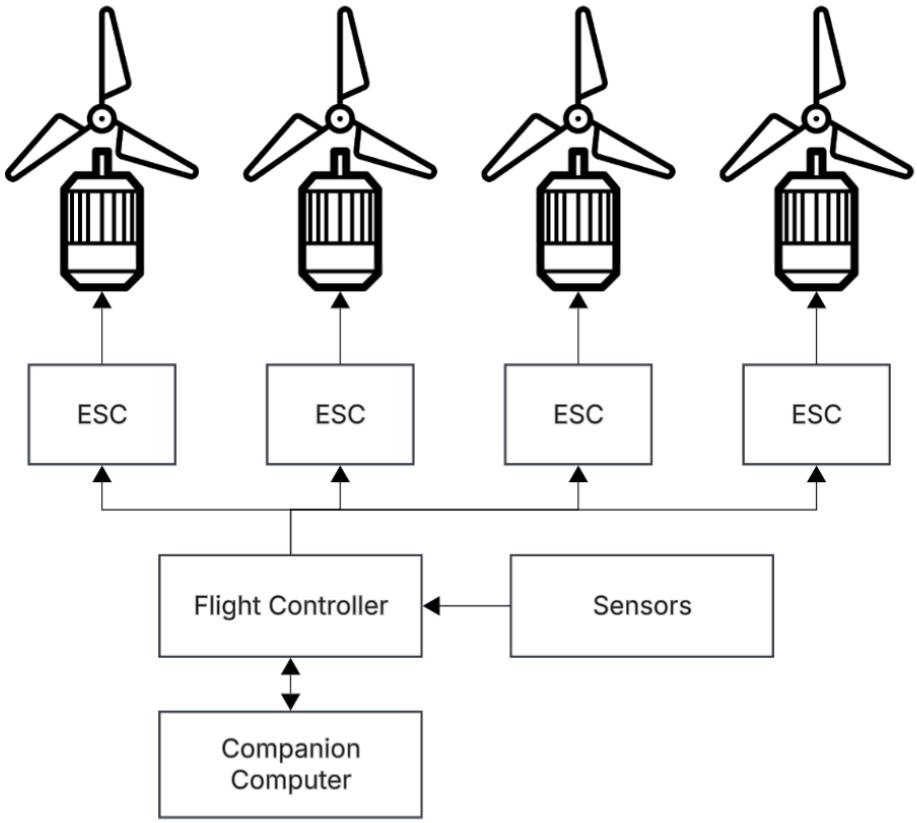

Figure 1: Diagram of quadcopter system.

At the lowest level, a UAV consists of a few basic components. Brushless motors spin propellers to generate upwards thrust on the vehicle frame. The speed of the motors is controlled by electronic speed controllers (ESCs), dedicated hardware that translates digital commands into

motor outputs. A suite of sensors including an inertial measurement unit (IMU), barometer, and GPS tells the system where in the world the vehicle is, how fast it is moving, and in what direction. Tying all of this together is the flight controller (FC), the core processing unit that reads sensor data, runs stabilization algorithms, and commands the ESCs to keep the drone airborne. Flight controllers run software known as an autopilot, with PX4 being a prominent open-source example, which abstracts the low-level hardware into a set of control interfaces. These interfaces allow external systems to issue movement commands, providing the foundation on which higher-level autonomy can be built.

This is where the companion computer comes in. While the flight controller is solely responsible for vehicle stabilization and low-level motion, a companion computer sits alongside it and interfaces with it to provide that higher-level autonomy. Tasks such as perception, mission logic, and decision-making are computationally intensive and do not need to operate at the same tight timing requirements as the flight control loop, making them well suited to run on a separate, more capable machine. The companion computer issues high-level commands to the flight controller through its exposed interfaces, which then translates those commands into actuator outputs. This division of responsibility for autonomous drone control is well established in practice, however information regarding how to build the companion computer side of this system remains limited in the open-source ecosystem.

PX4, the autopilot software, and ROS2, a robotics software framework, are both well-documented individually. PX4 provides thorough documentation for its flight modes, failsafe behavior and Offboard control interacts [1]. ROS2 provides documentation for middleware, communication and tooling [2]. The code examples shown in each one of these documentations are minimal and only present that communication between these two systems is possible. The

gap between a minimal working example and a production-ready system is significant and largely undocumented. What these examples do not cover, and what a real-world system requires, is how to monitor system health, how to classify and respond to faults, how to explicitly authorize when the companion computer is allowed to control the drone, and how to separate mission logic from safety-critical logic.

This gap is particularly consequential for aerial vehicles. Unlike ground robots where a safe idle state is as simple as stopping in place, a UAV must maintain active flight even during fallback conditions, meaning that the logic for recovery requires more careful handling than simply stopping in place. Although the PX4 autopilot will default to a safe state when it detects a FC level failure, it has no visibility into the health of the companion computer itself. The responsibility of detecting and responding to these conditions falls entirely on the architecture of the companion computer itself. Most systems that do address these concerns are proprietary, Skydio or Anduril being notable examples, and well-documented open-source implementations of such systems are scarce. This leaves developers and researchers without a reusable reference, forcing teams to resolve the same foundational problems independently for each new use case. This project aims to address that gap by providing an open reference architecture and companion computer framework that treats supervision, fault handling, and safety as first-class design concerns, giving researchers and developers a modular framework to build off on.

Background Research

PX4 Autopilot

PX4 is an open-source autopilot software which is widely used in both research and commercial UAV systems. [1] This software runs on a dedicated flight controller and is responsible for the real-time, safety-critical operations of the vehicle such as reading sensor data, state estimation, and commanding actuators. PX4 is designed to operate at high frequencies with control loops running in the range of hundreds of hertz making it well suited for the time critical demands of flying drones. PX4 is intentionally scoped to low-level vehicle control and structured mission execution. PX4 can execute pre-planned waypoint missions uploaded from a ground control station (GCS), but is not designed to perform real-time perception or autonomous decision-making. These higher-level functions are instead delegated to an external system such as a companion computer. PX4 does expose a set of interfaces through which external systems can interact with it, the most relevant of which for companion computers is Offboard mode.

PX4 operates in discrete flight modes, each of which defines which source has setpoint authority over the vehicle at any given time. Offboard mode is the relevant one in this case and allows for an external system to send position, velocity, or attitude setpoints over a communication interface. Most importantly, PX4 always retains override authority, so if the external setpoint stream is lost, RC override is detected, or an internal failsafe condition is triggered, then PX4 will immediately exit Offboard mode and transition into a predefined safe state. To maintain the autopilot in Offboard mode requires a 2Hz handshake between the FC and the external system (companion computer). This authority model means that the control that the companion computer has is always revocable and under advisory.

ROS2

ROS2, which stands for Robot Operating System, is an open-source middleware framework widely used in robotics research and development. [2] Software built with this framework follows a distributed publisher-subscriber communication model in which individual processes called nodes communicate by publishing and subscribing to names topics. This architecture allows for systems to be built from individual modular components that can be developed and tested in isolation. ROS2 is a natural fit for companion computer software for several reasons. Its node-based model maps well to the layered design of an autonomy stack, where supervision, control, and decision making can be implemented as separate nodes with well-defined interfaces. The topic system provides a nice, standardized way to pass data between components of the system without tight coupling. Additionally, ROS2's support for lifecycle nodes and quality of service policies provides the tooling necessary to build systems that are manageable at runtime. These features are critical for drone development where nodes must be started, monitored, and stopped in a controlled and predictable order, and where communication failures must be detected and handled and not ignored.

Behavior Trees

Behavior trees are a hierarchical, modular data structure for building decision-making logic in autonomous systems. [7] Originally developed for video games where you would want to define behavior for some monster or enemy, they have become a very popular paradigm for robotics behavior, replacing finite state machines. This is because behavior trees offer advantages in composability, readability, and ability to handle priority-based decisions in a structured and predictable way. A behavior tree is composed of nodes arranged in a tree-like structure. Leaf nodes represent actions or conditions, while the nodes in the middle define control flow. The tree is ticked at a fixed rate, with each tick propagating from the root and

returning a status of success, failure, or running from each node. This model allows for higher priority safety behaviors to come before lower priority mission tasks, making behavior trees well suited to the decision layer of a autonomy stack where mission objectives must always yield first to safety constraints.

Existing Solutions

Aerostack2

Aerostack2 is a popular open-source framework for autonomous drones systems built on ROS2 and designed for multi-robot aerial operations. [3] It features platform independence through plugin-based hardware abstraction, a modular component architecture, and behavior tree-driven mission control, making it one of the more mature frameworks in the open-source ecosystem. It supports multiple different flight stack hardware and vehicle configurations. Aerostack2 is also well used across a range of academic and research deployments and provides a lower barrier of entry for teams seeking rapid development. However, Aerostack2's design priorities also introduce certain limitations. Its emphasis on platform generality and breadth means that safety enforcement is not treated as a first-class design concern. The framework does not define a formal authorization mechanism of governance when the companion computer is permitted to influence the vehicle versus flight controller in safe modes. This limits it suitability in use contexts where safety assurance must be structurally enforced and not prioritized equally to application-layer programs. Since Aerostack2 is distributed as a complete framework, its architecture is difficult to partially adopt as teams with divergent design require may find it challenging to pick and choose integration of certain components.

Nav2

Nav2 is among the most mature and widely adopted autonomous navigation frameworks in the ROS2 ecosystem. [4] Specifically designed for ground robotics, it provides a comprehensive navigation stack which includes path planning, obstacle avoidance, cost map management, and behavior tree-based mission execution. Nav2's broad adoption across both research and commercial ground robotics demonstrations that its design patterns are robust and well-validated in practice. Nav2's design assumptions are rooted in the physical characteristics of ground vehicles, with naturally imposes a lot of limitations when looking at it in the context of aerial robotics. A ground robot that encounters a fault can decelerate and stop place, allowing the system to recover or wait for operator intervention without much consequence. However, for UAVs, there's no equivalent passive safe state that exists. An aerial vehicle must actively manage its behavior during any fault condition, whether returning home, executing a controlled descent, or alerting an operator. UAVs require more explicit and structured supervision than what Nav2's architecture provides. The absence of a comparable framework addressing these aerial-specific supervisory concerns in the ROS2 ecosystem represents a significant gap that ground-oriented stacks just as Nav2 are not designed to fill.

MAVSDK

MAVSDK is a set of libraries that enable developers to create software applications that interact with MAVLink-based vehicles, providing a consistent API regardless of the underlying hardware or communication protocol. [6] It is designed to run efficiently in embedded setups, including onboard a drone on a companion computer, while providing a simple type-safe interface. Its plugin-based architecture covers common flight operations including telemetry and mission execution, and it supports bindings across multiple programming languages. MAVSDK's design scope is intentionally narrow. It functions as a vehicle communication

library rather than an autonomous systems framework and does not provide the higher-level components required for autonomous operation such as behavior management, state estimation, or fault supervision. While many features work on other flight stacks, implementation differences at the MAVLink microservices level mean that not every API will function correctly outside of PX4. MAVSDK exposes MAVLink services to the application code but defines no architectural model for how autonomous decisions should be structured. This makes it a useful building block for drone software development but does not guide how the system itself should function.

Architecture Design

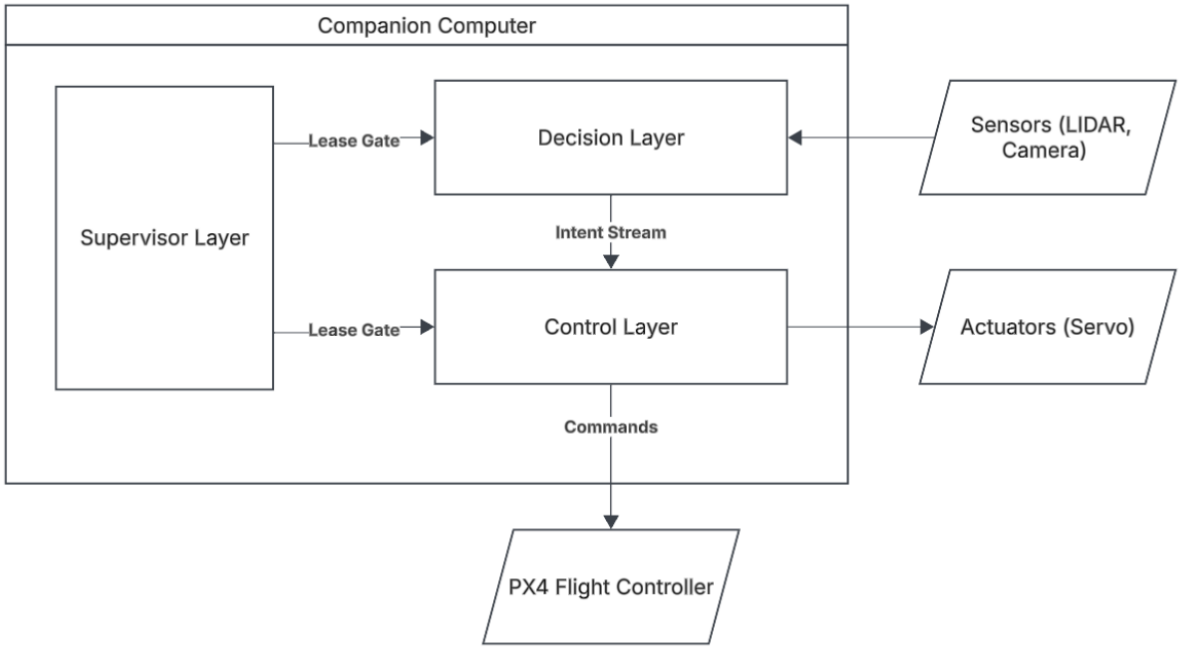

Figure 2: Diagram of UAV Lease-Gated Autonomy Stack.

The UAV Lease-Gated Autonomy Stack is a companion computer software architecture designed around a single core philosophy: autonomy must always be explicitly authorized and never assumed. This principle addresses one of the most important challenges of aerial autonomy. A UAV operating in a degraded or unauthorized state cannot simply stop flight as it needs to continuously manage its behavior. The architecture takes inspiration from established patterns in ground robotics, specifically the layered separation of concerns seen in frameworks such as Nav2 but adapts them to the constraints of aerial vehicles operating under PX4's revocable authority model. The stack is organized into three decoupled layers, the Supervisor Layer, the Control Layer, and the Decision Layer, each with a distinct responsibility and well-defined interfaces between them. This separation ensures each layer can be developed, tested, and modified in isolation, with no single layer able to bypass the responsibilities of another. Connecting these layers is the lease gate, a time-bounded authorization barrier that enforces the

core philosophy of the architecture. Only when a valid lease has been granted by the supervisor is decision layer intent permitted to pass through to the flight controller.

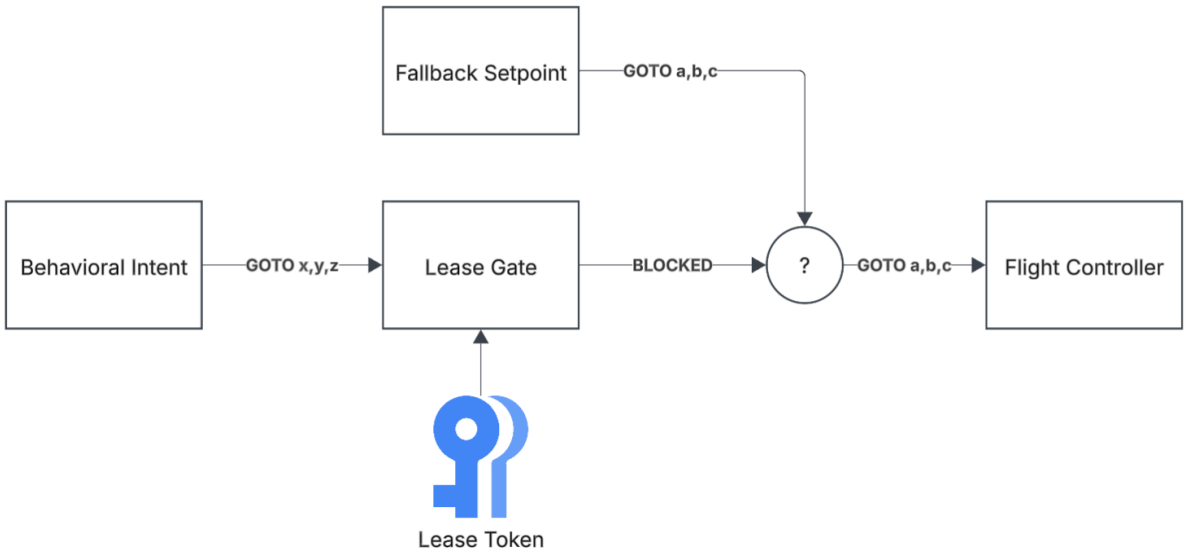

Figure 3: Diagram of lease gate mechanism

The lease gate is the primary enforcement mechanism of the architecture, operating as a barrier between the decision layer and the flight controller. It maintains two states: granted, where intent from the decision layer is authorized to pass through the control layer to the flight controller, and revoked or expired, where decision layer intent is blocked and the control layer falls back to safe hold setpoints. A lease is a short-lived token issued by the supervisor only when a strict set of safety preconditions are satisfied, meaning authorization is time-bounded and must be continuously renewed. It is important to note that the lease exclusively gates commands that originated as intent from the decision layer. The control layer always maintains the PX4 Offboard handshake and retains the ability to publish safe fallback setpoints regardless of lease state, ensuring the vehicle remains in a controlled and predictable state even when the supervisor has not authorized decision layer intent.

The Supervisor Layer is the ultimate authority for autonomous operation in the stack. It sits above both the Control and Decision Layers in terms of authority, continuously monitoring the entire system and deciding if and under what conditions is autonomy permitted. The supervisor's responsibilities are divided across three components: the Lease Manager, the Health Monitor, and the Fault Handler. The Lease Manager is responsible for granting, renewing, and revoking leases to the control layer based on a set of predefined safety conditions, and is the supervisor's primary mechanism for controlling whether decision layer intent is permitted to influence the vehicle at any given time. The Health Monitor continuously verifies the liveliness of the control and decision layers through heartbeat monitoring, watches for PX4 failsafe flags, monitors control loop timing, and tracks hardware resource usage on the companion computer (CPU and Memory). When any monitored condition falls outside of acceptable bounds, a fault in generated and passed to the Fault Handler. The Fault Handler is responsible for processing faults within the system by first classifying it against predefined severity levels and execute the corresponding safety directive to the control layer. Directives are high priority commands that instruct the control layer to either transition to a specific PX4 mode, publish fallback setpoints, or ignore decision layer intent until fault is cleared. Directives take absolute precedence over any intent coming from the decision layer. The supervisor has no direct communication with the flight controller and must enforce all safety actions through the control layer.

The Control Layer is responsible for converting decision layer intent into PX4 compatible flight commands when authorized through a valid lease. It serves as the interface between the companion computer and the flight controller, which means that all commands to the flight controller, whether they originate from decision intent or supervisor directions, pass through the control layer. The Intent Validator applies deterministic safety filters to all incoming commands,

making sure they remain within the vehicle's physical limits, geofence boundaries, and operational constraints before they are sent to the flight controller. Directives from the supervisor bypass intent validation entirely and take absolute precedence over any decision layer intent, as they are critical safety instructions that must be executed immediately. The FC Offboard Interface maintains the PX4 Offboard handshake at the require minimums rate of 2Hz, and executes a fixed rate control loop, publishing safe hold setpoints whether the lease is a valid and safe command stream regardless of the state of the layers above.

The Decision Layer is responsible for determining what the vehicle should do based on mission objectives, sensor data, and environment context. It provides intent only, meaning it has no direct command authority over the flight controller and all output must pass through the control layer's validation and lease gating before it can send commands to the flight controller. The decision logic is implemented as a composable behavior tree, where individual behaviors are defined as leaf nodes and the control flow is managed through central nodes that determine how and when each behavior executes. A developer implementing their own mission logic does so by building their own behavior tree nodes that produce intent, which are then composed into the tree alongside existing safety and navigation behaviors without modifying any other parts of the stack. Higher priority safety behaviors are of course placed higher in the tree and can preempt lower priority mission tasks, ensuring that mission objects always yield to safety constraints in a predictable way. The decision layer is also responsible for producing a single intent stream, keeping arbitration logic constrained within the behavior tree rather than distributed across the different layers. This single intent stream simplifies the control layer's responsibilities and ensures that the interface between the decision and control layers remains clean and well-defined.

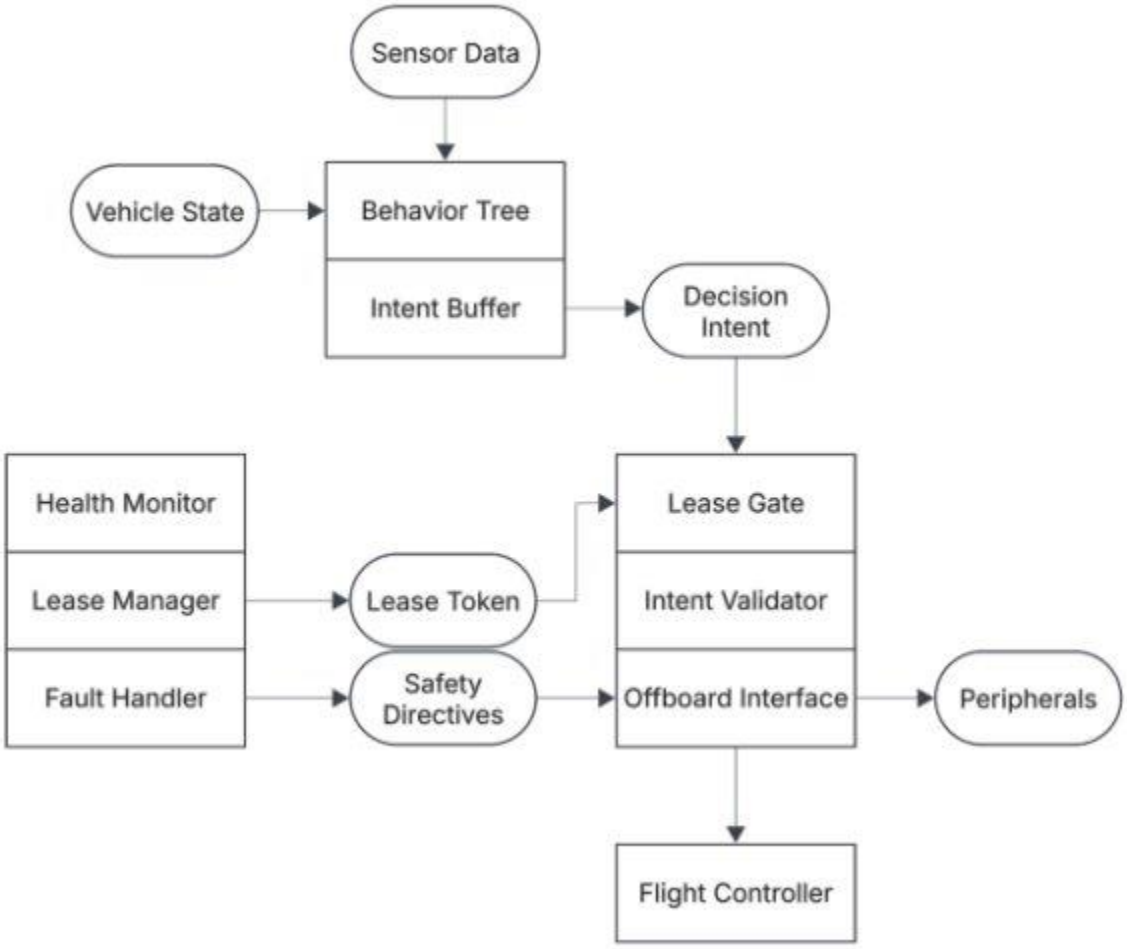

Figure 4: Flow of Command through the stack

To help understand how these layers interact, consider the full flow of a command through the stack. The behavior tree in the decision layer ticks to produce an intent, such as moving to a target position or changing to a specific flight mode, which is passed as a stream to the control layer. The control layer checks the lease state and if it's valid, the intent passes through the intent validator which applies safety and envelop checks before forwarding the command to the flight controller via the Offboard interface. If the lease is revoked or expired, the intent is blocked at the gate, and the control layer publishes safe hold setpoints instead, while maintaining the offboard handshake with the PX4 flight controller. Throughout this journey, the

supervisor layer is continuously monitoring system health, PX4 state, and node liveliness, ready to revoke the lease or issue a directive to the control layer the moment a fault condition is detected. Together, these three layers and the lease gate form an autonomy stack where safety enforcement is a structural requirement, and where mission logic, control enforcement, and supervision are cleanly separated so that each can be extended or replaced without compromising the integrity of the others.

Developer Workflow

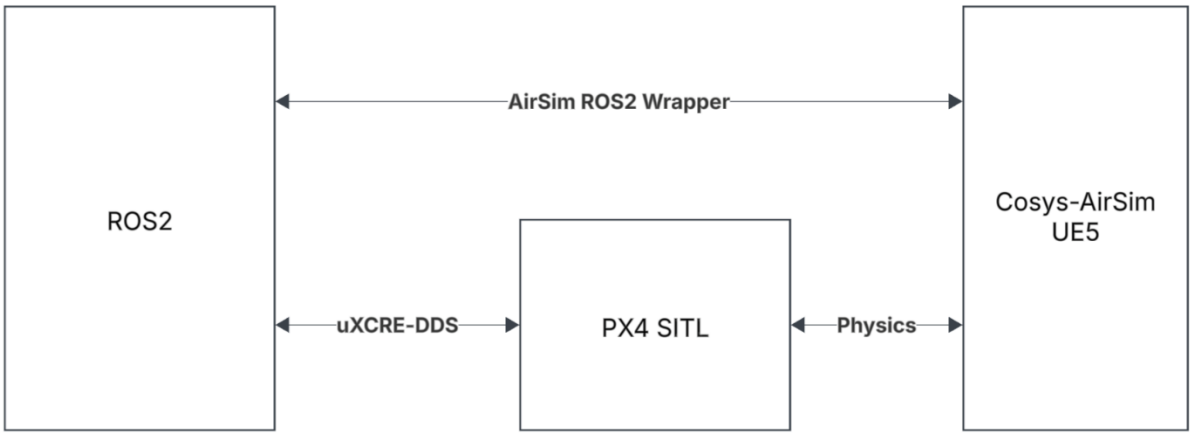

Figure 5: Diagram of software simulation stack for development.

Setting up the development environment is the first step for developers looking to implement or extend this architecture. Cosys-AirSim is an open-source simulator built on Unreal Engine 5 that provides high-fidelity physics and sensor simulation for aerial vehicles. [8] PX4 SITL (Software In The Loop) runs the full PX4 autopilot software in a simulated environment, replicating the behavior of a real flight controller without the need for physical hardware. [9] Together, these two tools form the simulation stack used in this project, allowing developers to iterate on companion computer logic and validate behavior against a realistic flight model before committing to hardware. The AirSim ROS2 wrappers expose simulator-level data like sensor

readings and vehicle state as standard ROS2 topics to be consumed by the companion computer. Communication between ROS2 and PX4 is handled by the uXRCE-DDS bridge which exposes PX4's internal uORB messages as ROS2 topics, ranging from changing flight modes to setpoint commands. This setup provides the necessary communication interfaces needed to start implementation in ROS2 and allows for easy startup and tear-down during iteration.

Figure 6: Cosys-AirSim plugin for Unreal Engine 5

Each of the three architectural layers should be implemented as one or more ROS2 nodes, with well-defined topics serving as the interfaces between them. For example, the intent stream through which the decision layer passes behavioral intent to the control layer, should be published on a dedicated topic using best-effort QoS to minimize latency. Lease state and supervisor directives should use reliable QoS to ensure that messages aren't dropped between the supervisor and control layers. All nodes in the stack should be implemented as lifecycle nodes, which provides a standardized state machine for managing node startup and shutdown in a controlled and predictable order. The supervisor-related nodes must be fully active before the

control and decision layers are permitted to start. A single ROS2 launch file should bring up the entire stack in the correct sequence, starting with the supervisor first, then the control layer, and finally the decision layer. This order ensures that the lease gate and health monitoring are in place before any mission logic begins executing.

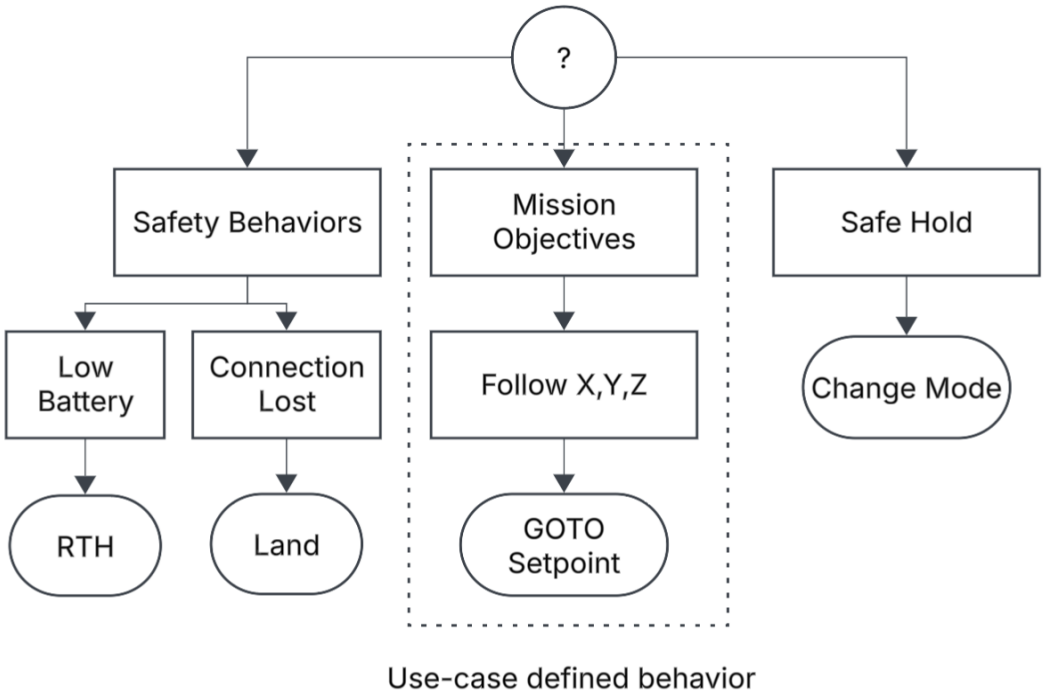

Figure 7: Structure of the decision layer's behavior tree.

The final implementation concern is the decision layer's mission logic. Behavior trees are the model used to produce intent for how the drone should behave. The tree is structured around a top-level priority selector that evaluates safety behaviors before any mission logic, meaning conditions like low battery, link loss, or geofence violations will always preempt the current mission task. Each safety condition is implemented as a leaf node that reads from the relevant ROS2 topic, such as vehicle status or failsafe flags, and returns a failure status when the condition is not met. Mission tasks themselves are composed of subtrees, each encapsulating the

actions and conditions required to complete a mission objective. This composability allows new mission behaviors to be added without modifying the safety layer, since the priority selector structure guarantees that safety checks are always evaluated first. The behavior tree ticks at a fixed rate, producing a single intent command per cycle that is forwarded to the control layer for validation and gating.

Discussion

The goal of this architecture is to address the gap between the minimal examples found in these software frameworks, such as ROS2 or PX4, and a production-ready companion computer by providing a structured architecture where safety enforcement is a fundamental structure of the design. The lease-gate mechanism is the main driver of this goal by making the action of acting autonomously explicit, time-bounded, and enforced. Through this, the architecture ensures that the decision layer can never influence the vehicle outside of a known safe envelope regardless of what mission logic wants the drone to do. The three-layer separation holds up well in simulation, with each layer maintain a clearly bounded responsibility and the interfaces between them remaining clean and well defined. The architecture also aligns well with PX4's authority model, working with its revocable control paradigm, which means that safety of the flight controller and the safety guarantees of the companion computer reinforce each other rather than conflict. Taken together, the architecture achieves its stated design goals and stands as a documented reference that researchers and developers can study, adopt, and build on without having to solve the same foundational problems from scratch.

The most significant limitation of this work is that the architecture has only been validated in simulation and has not been tested on physical hardware. While Cosys-AirSim provides high fidelity rendering and physics simulation, hardware specific factors such as

communication latency, sensor noise, and PX4 tuning between simulated and real platforms have not been accounted for. Real world validations remain a necessary next step before the "production-ready" claims can be properly backed up. Another limitation is that several of the architecture's safety components are implementation defined. Some examples include lease preconditions, fault severity levels, and directive response policies which are left to the developer to implement. This means that the safety guarantees are only as strong as the implementation choices. The architecture is also explicitly scoped to single drone systems and does not address multi-vehicle coordination. Finally, the supervisor communicates with the flight controller indirectly through the control layer interface rather than directly, which is a deliberate design choice that separates responsibilities between the layers; however, this decision could add additional overhead in the fault response path for time-critical fault events.

This work offers several directions for future iteration. Hardware validation on a physical PX4 platform is the most immediate next step, as real-world deployment will definitely unearth some considerations that simulation alone cannot capture. Expanding the documentation of the supervisor's fault classification system would strengthen implementation as it currently relies on developer judgement. Another possible path is extending the lease-gate concept to multi-vehicle systems, exploring how autonomy authorization could be coordinated across multiple drones without compromising the safety guarantees of individual vehicles. These directions represent a natural next step to transition from a reference architecture to a more complete framework, each building on the foundation that this work establishes.

Conclusion

This paper presented the UAV Lease-Gated Autonomy Stack, an open reference architecture for companion computer software on PX4 based autonomous UAV systems. The

architecture addresses a gap in the open-source ecosystem by providing a structured, documented foundation that treats supervision, fault handling, and safety enforcement as first class design concerns rather than afterthoughts. By organizing the system into three decoupled layers unified by an explicit lease gate mechanism, the architecture ensures that autonomy is always deliberately authorized and that safety guarantees are structural properties of the system rather than implementation dependent checks. It is the hope of this work that the architecture and its accompanying reference material serve as a useful foundation for researchers and developers building autonomous UAV systems, lowering the barrier to safe and reproducible companion computer development.

References

[1] “PX4 User Guide”, *docs.px4.io.* https://docs.px4.io/main/en/

[2] “ROS 2 Documentation – ROS 2 Documentation: Humble documentation,” *docs.ros.org.* https://docs.ros.org/en/humble/index.html

[3] "Aerostack2 Documentation," *aerostack2.github.io.* https://aerostack2.github.io/

[4] "Nav2 Documentation," *nav2.org*. https://nav2.org/

[5] "CTU MRS UAV System," *ctu-mrs.github.io*. https://ctu-mrs.github.io/

[6] "MAVSDK FAQ," *mavsdk.mavlink.io*. https://mavsdk.mavlink.io/v2.0/en/faq.html

[7] G. Dromey, "Behavior Trees: From Systems Engineering to Software Engineering," in *2010 8th IEEE International Conference on Software Engineering and Formal Methods*, Pisa, Italy, Sep. 2010, pp. 13–18. https://ieeexplore.ieee.org/document/5637403/

[8] "Cosys-AirSim Documentation," *cosys-lab.github.io*. https://cosys-lab.github.io/Cosys-AirSim/

[9] "PX4 Software-in-the-Loop (SITL) Simulation," *docs.px4.io*. https://docs.px4.io/main/en/simulation/